ChatGPT

1. 核心问题定义 (Core Problem Definition)

Agent 定义 (Definition of "Agent"): 从广义上讲，**Agent（智能体）**指能够自主感知环境并采取行动以实现目标的智能系统 [1] [2]。传统定义强调代理的自主性、环境感知和行动能力。例如，Wooldridge 和 Jennings 将 Agent 定义为一个具备 **自主性**（可在无人工干预下自主运行）、**社会能力**（能通过通信语言与其他 Agent 或人交互）、**反应性**（能感知环境变化并及时响应）以及**前瞻性**（具备主动的目标导向行为）的系统 [1] [2]。早期 Agent 通常通过传感器感知环境状态，经内部策略决策后，通过效应器执行动作 [3]。在认知架构中，Agent 被视为具有感知-决策-行动循环的认知实体，具备记忆、推理等模块；而在现代大型语言模型 (LLM) 驱动的 Agent 中，预训练模型本身扮演了决策引擎的角色，能够解析自然语言指令并产生行动方案 [4] [5]。

认知架构与语言模型代理: 认知架构（如 Soar、ACT-R）是早期用于模拟人类认知过程的 Agent 框架，包含感知、记忆、推理等固定组件 [6]。例如 Soar 架构旨在提供构建广义智能 Agent 的计算模块，支持决策、问题求解、规划和自然语言理解等能力 [6]。BDI 模型则以**信念-愿望-意图**来描述 Agent 的心理状态，指导其计划与行动，被认为是实践推理 Agent 的经典模型 [7]。相比之下，新一代基于大型语言模型的 Agent 利用预训练模型的知识和推理能力，通过提示（prompt）来控制行为。**语言模型代理**能够理解人类自然语言指令，并在无需明确编程规则的情况下，依托LLM的生成能力进行**链式思考和行动**（chain-of-thought + act） [8]。这类 Agent 常通过在提示中交叉插入"思考（Thought）"和"行动（Action）"来进行问题求解 [8]。语言模型代理的优势在于通用性强，可通过少量示例掌握新任务，但也受限于模型知识截止和可靠性问题。

工具协同与环境交互: 现代 Agent 一个显著特点是能够调用外部工具或服务来增强能力 [9]。**工具协同**是指 Agent 在完成复杂任务时，可以自主决定调用何种外部API或工具，如搜索引擎、计算器、数据库查询等，以弥补自身知识或算力的不足 [10]。例如 Toolformer 提出让语言模型通过自监督学习学会调用API，在需要时自动查询工具并利用结果 [11]。同时，Agent 也可以与环境直接交互，这里的 **环境交互**既包括虚拟环境（如网页、操作系统界面）也包括物理环境（如机器人与物理世界的互动）。在虚拟环境中，Agent 可以执行浏览网页、点击按钮等操作，从而实现如自动网页浏览、GUI 自动化操作等应用；在物理环境中，Agent 则通过机器人执行移动、抓取等动作。环境交互的紧密程度因 Agent 类型而异：有的 Agent 完全工作在语言空间，通过问答完成任务；有的 Agent 则深度耦合环境，例如嵌入式机器人 Agent 持续感知物理世界并做出响应。具备多模态感知（如视觉、语音）的 Agent 可以从图像、语音等多模态输入中获取信息，丰富了环境耦合程度，使 Agent 在复杂环境中也能自主决策和行动 [12]。

Agent 分类体系: 可以从多个维度对 Agent 进行分类 [13] [1]：

- 按自主性：从**辅助型Agent**（在人类指令后执行单步操作）到**全自主Agent**（给定高层目标后可自行分解任务、连续行动）。自主性越高，Agent 越能在无监督下长时间自主运行。
- 按通信能力：区分**单体Agent**（无需与其他体通信）和**多智能体**（能够与其他Agent协作或竞争，通过消息通信完成任务）。具备通信能力的Agent可以组成多智能体系统，实现分布式任务求解 [1]。
- 按环境耦合程度：从**静态环境Agent**（仅基于离线数据或静态知识作答）到**动态环境Agent**（实时感知环境变化并做出响应）。例如，仅通过文本问答的Agent环境耦合度低，而嵌入式机器人、网页代理等实时交互的Agent环境耦合度高 [3]。
- 按多模态能力：区分**单一模态Agent**（仅处理文本对话）和**多模态Agent**（能处理文字、图像、语音等多种模态输入输出）。多模态Agent在需要视觉理解或语音交互的场景下表现更好。

上述分类维度并非彼此独立，一个 Agent 往往在多个维度上呈现不同特性。例如，一个高度自主、能通信协作的多模态机器人 Agent，在分类体系中兼具多重属性。构建 Agent 分类体系有助于我们理解不同 Agent 架构的异同，并据此选择合适的技术路径来实现目标功能 [14] [15]。

主流 Agent 架构对比: 当前有多种主流 Agent 架构和实现，它们在机制上各有侧重：

- **ReAct 框架：** 这是一种将“推理 (Reasoning)”与“行动 (Acting)”交替结合的通用Agent范式 [8]。ReAct通过提示模版，引导LLM一边生成思考链条，一边在需要时输出动作指令（如调用工具）。其机制适合需要边思考边与环境交互的问题，例如回答开放性问题时搜索资料 [8]。ReAct 框架的优势在于**灵活性**和**通用性**，目前被 LangChain、AutoGPT 等众多系统所采用 [8]。但ReAct也需要设计良好的提示来避免Agent进入无效循环。
- **AutoGen 平台：** AutoGen 是微软研究提出的一个用于构建多Agent对话工作流的框架 [16]。它将复杂任务表示为多个专长Agent之间的**对话协作**。AutoGen 提供可定制的可对话 Agent，以及简洁的多Agent编排接口，使开发者通过定义角色和交互规则即可组装出复杂能力 [17] [18]。与 ReAct 不同，AutoGen 更像是一个**多Agent体系**：开发者可以指定若干 Agent（基于LLM、人类或工具）及其交互逻辑，这些 Agent 以聊天形式沟通完成任务。AutoGen 适用于将任务拆解给不同专长 Agent 协作解决的场景，例如一个Agent负责规划，另一个Agent负责执行代码等。
- **LangGraph 架构：** LangGraph 是 LangChain 生态中新推出的开源 Agent 框架，采用**有向图**形式编排多Agent工作流 [19]。与以往线性链式调用不同，LangGraph 允许开发者显式定义各 Agent 节点及它们之间的转换关系，形成可控的图结构。这使得复杂多步骤流程可视化为状态图，提供了**状态持久**和**可观察**的 Agent 运行机制 [19] [20]。LangGraph 强调对 Agent 状态和决策路径的透明管理，内置“状态”作为记忆单元记录过程数据，方便调试和理解 Agent 决策 [21] [20]。它适合需要精准控制多Agent协作流程的应用，例如 IT 运维支持场景下，不同Agent按图定序协作 [19]。
- **OpenAgents 平台：** OpenAgents 是一个开放平台，旨在让开发者和普通用户都能方便地使用和部署语言 Agent [22] [23]。OpenAgents 提供了图形界面和三类内置 Agent：(1) 数据Agent（使用 Python/SQL 进行数据分析）；(2) 插件Agent（集成200+常用API工具）；(3) Web浏览Agent（执行自主网页浏览） [24]。用户通过 Web UI 与这些 Agent 交互，可实现数据查询分析、调用在线服务以及网页信息收集等日常复杂任务 [23]。OpenAgents 的特点是**易用性**和**通用性**，在用户体验层面屏蔽了复杂的提示工程，让非专家也能使用 Agent功能 [25]。其机制侧重于提供一个统一门户来访问多种Agent功能，并能在本地部署以保护数据隐私 [26]。

综上，不同 Agent 架构在自主决策、协作方式、记忆结构和应用场景上存在差异。ReAct 强调单Agent的推理-行动闭环，AutoGen/LangGraph 强调多Agent分工与工作流编排，而 OpenAgents 注重平台易用性和工具集成。根据具体应用需求，可选择适合的架构：例如问答搜索可用 ReAct 模式，软件开发可用多Agent协作模式等，在实践中甚至可以将多种范式混合应用，以兼顾推理深度和操作复杂度。

2. 学术界脉络 (Academic Developments)

认知架构与 BDI 演化: 智能 Agent 的研究可以追溯到 20 世纪末的认知架构和 Agent 理论。早期工作聚焦于模拟人类智能的结构，例如 **Soar** 和 **ACT-R** 等认知架构 [6]。Soar 项目旨在构建通用智能体的基本模块，使 Agent 具备人类般的决策、问题求解、规划和自然语言理解能力 [6]。**ACT-R** 则从认知心理学出发，模拟人的记忆和认知过程，强调与心理实验的一致性。与此同时，**BDI (Belief-Desire-Intention)** 模型在 1990 年代形成，它将 Agent 的内部状态抽象为“信念-愿望-意图”，提供了一种形式化的方法来描述 Agent 的决策过程 [7]。BDI 模型认为 Agent 根据其对环境的信念、想要实现的目标（愿望）以及选择执行的计划（意图）来行动，是实践推理 Agent 的典型框架。这些理论为多智能体系统奠定了基础 [27] ——例如 Castelfranchi 和 Ferber 等人在 1990 年代提出了 Agent 社会行

为和分布式协作的本体分类，强调自主 Agent 的交互可以涌现复杂的群体行为 [27]。总体而言，学术界在该阶段确立了“Agent = 自主+感知+行动+通信”的共识，并发展出相应的软件架构与编程语言，为后续 Agent 研究提供了方法论指导。

关键论文方法 (近年): 进入深度学习时代后，尤其是 2022 年以来，大型语言模型赋能的 Agent 成为研究热点。一系列具有里程碑意义的论文提出了创新的方法来提升 Agent 的推理、工具使用和规划能力：

- **ReAct (Yao et al., 2022)：** 这项工作提出了将**链式思考**与**动作执行**融合的范式 [8]。ReAct 框架允许 LLM 在回答问题时，先生成多步的思考过程，然后在需要时发出动作指令（如查询数据库或调用API），再将结果纳入后续推理。通过在提示中提供示例，LLM 学会了交替输出“Thought:”和“Action:”序列。这种 **Reason + Act** 的机制让模型一方面具备了解释性推理链，另一方面又能实际与环境交互，从而显著提升了在开放领域问答、推理任务中的表现 [8]。ReAct 框架开创了用 LLM 控制工具的先河，其思想被后续众多工作借鉴，成为构建复杂Agent系统的基石。
- **Toolformer (Schick et al., 2023)：** 工具使用能力对 Agent 很关键。Toolformer 提出让语言模型**自我监督地学习调用工具** [9]。作者设计了一种方法，让预训练语言模型在大量无标注文本中插入调用API的标记作为训练数据，从而得到一个 fine-tune 后的模型，它可以自主决定何时调用何种工具API以及如何利用返回结果 [11]。例如，它教会模型使用计算器API完成算术运算，或使用检索API查询知识 [28]。Toolformer 的贡献在于免除了人工设计提示或规则，让模型**内化**了工具使用策略，实现零样本下性能提升 [10]。这证明了大型语言模型可以通过适度微调，掌握调用外部工具的技能，为后续 Agent 集成各种工具提供了参考范式。
- **Tree of Thoughts (Yao et al., 2023)：** 针对复杂问题求解，Tree-of-Thought (ToT) 将传统 AI 中的“搜索树”概念引入语言模型推理 [29]。不同于 Chain-of-Thought 线性推理，ToT 让 Agent 在每一步可以**分叉产生多个想法**，形成树状搜索空间，并具备回溯和评估机制 [30]。具体而言，LLM 会生成一系列可能的中间推理步骤（thoughts），然后评估哪些分支有望通向正确解，再有选择地向下展开，从而克服了贪心生成单一路径的局限 [31]。通过这种 deliberate reasoning，Agent 可以在解谜、规划等需要搜索的任务上取得更高成功率。例如在谜题和数学题中，ToT 比单一路径的CoT取得了显著更高的正确率。该方法展示了通过**启发式搜索**扩展LLM推理能力的新方向，将搜索算法与语言模型有机结合。
- **Reflexion (Shinn et al., 2023)：** Reflexion 引入了**自我反思与反馈机制**来提升 Agent 的决策成功率 [32]。其核心思想是在每轮任务尝试后，让 Agent 反思自身行为，生成自然语言的自我反馈，并将其存入长期记忆，用于下一次尝试改进决策 [33] [32]。Reflexion 框架包含 Actor（执行当前策略）、Evaluator（评估结果）、以及 Self-Reflection 模块。具体过程是：Agent 根据当前策略产生一系列行动轨迹，Evaluator 打分或给出纠正提示，然后 Self-Reflection 模块将这些反馈总结为建议添加到记忆中 [34] [35]。在下一次任务执行时，Agent 会参考这些累积的反馈，避免重复过去的错误。实验表明，在决策类任务（如 ALFWorld 环境决策）、开放问答（如 HotPotQA）等场景中，引入 Reflexion 后，Agent 完成任务的成功率显著提升 [36]。例如，带 Reflexion 的 ReAct 在 ALFWorld 任务上完成130/134个目标，而纯 ReAct 仅完成较少数量 [36]。Reflexion 的出现标志着 Agent 开始具备某种元认知能力，通过自我纠错循环提高鲁棒性 [32]。这一思想也启发了更多“让模型反思模型”的后续研究。
- **其他代表性方法：** 除上述外，近年还有许多创新：如 **Generative Agents (Park et al., 2023)** 模拟了一个虚拟社会中多智能体的长期记忆和交互，在交互环境中展现类人行为；**HuggingGPT (Shen et al., 2023)** 则让一个语言模型充当“调度中枢”，调用 HuggingFace 上各种专家模型完成多模态复杂任务，展示了**大型模型+工具专家**的强大组合；**Self-Consistency (Wang et al., 2022)** 提出了对LLM的思考链进行多样采样并投票，选择最一致的答案，以提高稳定性 [37]；此外还有如 **Voyager (Wang et al., 2023)** 实现在 Minecraft 游戏中持续自主探索和技能学习的终身型Agent、**Mind’s Eye** 将视觉反馈引入LLM等。这些工作共同丰富了 Agent 的技术谱系，在推理、规划、工具使用、多智能体协作等方面不断拓展边界。

近三年顶会论文 (ACL/NeurIPS/ICLR/ICML/EMNLP): 由于 Agent 构建是多学科交叉热点，大型语言模型兴起后，各大 AI 顶会上出现了大量相关论文。以下列举近三年部分具有代表性的工作：

- *2021:* NeurIPS 2021 上出现了让语言模型生成代码并执行以解决问题的方法雏形；ACL 2021 讨论了对话代理的一致性维护和长期交互的问题；ICLR 2021 则开始探讨利用大模型进行规划的可能性。
- *2022:* ICLR 2022 的工作如 “Chain-of-Thought” 提出通过中间推理链提升模型复杂推理能力 [38] ；NeurIPS 2022 则出现了 ReAct 等论文（发表于ArXiv）探讨LLM与环境交互。EMNLP 2022 开始有工具使用（如检索增强）相关论文。
- *2023:* 这一年是爆发期。ACL 2023 出现了关于让LLM驱动**多智能体对话**协作完成任务的论文；EMNLP 2023 有多篇关于**Agent 自主性评估**、**任务分解**的研究；ICML/ICLR 2023 则关注**Planner-Executor架构**、**反思反馈**机制等主题，例如 ICLR 2023 收录了让LLM在机器人操作中进行高层规划 (LLM-Planner) [39] 。NeurIPS 2023 出现了 Reflexion [32] 、Generative Agents 等创新工作，多智能体模拟、内存机制等成为亮点。
- *2024:* 许多2024年发表或投稿的工作关注**Agent 安全**、**大模型代理评测**以及**标准协议**（如 MCP/A2A）的探讨。例如有论文系统性比较了现有 Agent 基准和评测框架 [40] [41] ，呼吁社区建立统一评测标准。
- *2025:* 虽然刚开始，但可以预见今年的研究趋势包括：如何让 Agent 更可靠地执行长链任务、如何引入显式的符号推理模块、Agent 如何进行自主的**长期学习**（lifelong learning）等等。这些都在持续演进中。

总的来看，近三年学术界在 Agent 研究上呈“井喷”态势。从让 LLM 学会调用工具、到引入搜索树和自我反思，再到多Agent协作和内存机制，研究者们着力弥补 LLM 的固有不足（如事实性、长期规划能力），以期让 Agent 更加智能、稳健和高效。这些前沿论文为工业界构建复杂 Agent 系统提供了思想宝库和技术储备。

经典评测基准:

- **ALFWorld:** 这是一个强化学习环境基准，用于测试 Agent 的决策和规划能力 [36] 。ALFWorld 基于 ALFRED 环境，Agent 接受文本形式的观察（房间描述和物体）并输出动作指令，目标是完成如搬运物品等家庭任务。Reflexion 等方法在该基准上大幅提高了任务完成率 [36] 。ALFWorld 评测 Agent 在连续决策、多步计划和错误纠正方面的能力，是验证自主 Agent 有效性的经典环境。
- **WebArena:** 一个用于评估 Agent **网页操作和信息检索**能力的现实环境基准 [42] 。WebArena 提供了仿真的网页界面（如购物网站、航班预订页面等），要求 Agent 像人一样通过浏览、点击、表单填写等完成指定任务（例如网上购物、预订行程） [42] 。它考察 Agent 的网页理解、跨页导航、信息抽取与规划能力，非常贴近真实互联网应用场景。2023 年的一篇论文详细介绍了 WebArena，并用其评估了不同 Agent 在这些日常任务上的表现 [43] 。
- **HotPotQA:** 尽管不是专门的 Agent 环境，HotPotQA 是学术界常用来测试多跳推理能力的问答数据集 [36] 。它要求系统通过阅读多个 Wikipedia 文段找到问题的答案，强调信息检索和综合推理。很多 Agent 技术（如 ReAct、Reflexion）都以 HotPotQA 来验证其改进的推理效果 [36] 。例如 Reflexion 帮助 Agent 减少错误推断，提高了 HotPotQA 多跳问答的正确率 [36] 。因此 HotPotQA 可以视作 Agent 工具使用（检索）和推理链质量的间接评测基准。
- **AgentBench:** 这是 2023 年提出的一个综合性 LLM Agent 基准框架 [44] [45] 。AgentBench 定义了八大类环境，覆盖**网页购物**、数据库操作、代码编辑等多种任务，让不同 Agent 在统一标准下执行比较 [46] 。它的出现旨在系统比较各类商用和开源 Agent 的性能 [46] 。AgentBench 的结果为社区提供了宝贵的比较数据，帮助找出现有方案的长短板。
- **ToolBench:** 专注于评估 Agent 的**工具使用**能力 [47] 。它提供了一套标准的 API 接口，让 Agent 在上面调用，以测试其选择和调用工具的正确性和效率 [47] 。ToolBench 覆盖计算、查询、翻译等常用工具使用场景，对比不同 Agent 在这些子任务上的成功率。它对于检验诸如 Toolformer 等模型的工具调用能力尤为适用。

- **其他:** 此外还有如 **BabyAI/Minigrid**（微型格网环境用于评估语言指令执行）、**Crafter**（2D 游戏环境测试生存策略）、**Minecraft** 开放世界环境等，都被用于不同类型 Agent 的能力验证。随着 Agent 研究推进，新的评测环境也不断出现，如 Facebook 提出的 **Diplomacy** 游戏环境、DeepMind 的 **OpenAI Arena** 等，用于评估多智能体博弈和协作。这些基准共同构成了 Agent 能力评测矩阵，从语言推理、工具使用、决策规划到多Agent交互，多维度刻画了Agent的智能水平。

随着研究的深入，评测基准也在迭代发展。例如最近有综述工作专门调查了 LLM Agent 的评测方法，比较了上述各种基准的覆盖能力和局限 [40] [41]。可以预见，将来会出现更贴近真实业务的复杂环境来评估 Agent，如模拟完整的软件开发流程或企业流程，从而推动 Agent 技术的改进和成熟。

3. 工业界技术演进与平台 (Industry Evolution and Platforms)

AutoGen 平台及演进: 工业界对 Agent 技术的探索中，**AutoGen** 是由微软开源的一个重要框架。AutoGen 将 LLM 应用抽象为由多个 Agent 对话协作完成任务的流程 [16]。开发者可以定义不同角色的 Agent（例如“用户”、“助手”、“工具”等），赋予它们不同能力，然后编排它们通过自动化对话完成复杂任务 [17] [18]。AutoGen 的出现标志着从单 Agent 向**多 Agent 系统**的转变，其提供的高层API让使用者专注于定义 Agent 行为逻辑，而无需手动处理底层 Prompt 切换和消息传递。微软不断更新 AutoGen —— 0.2 版本引入了更多模板，0.4 版本重构优化了代码以增强稳健性和可扩展性 [48] [49]。围绕 AutoGen 生态，微软还推出了 **AutoGen Studio**，这是一个**低代码/可视化的 Agent 构建界面** [50] [51]。借助 AutoGen Studio，用户可通过图形界面拖拽配置 Agent 及其交互流程，并能一键将设计好的工作流导出为 JSON 或部署为 API 服务 [52] [53]。这种能力实际上实现了 **Agent-as-a-Service**：企业可以将自定义的 Agent 工作流封装成API，集成到自有应用中 [52] [54]。AutoGen 的技术演进体现了工业界对**快速原型**和**易部署**的追求，从代码库升级到可视化Studio，再到与微软 Semantic Kernel 等工具集成，形成了一个完善的 Agent 开发与运行平台 [55] [56]。AutoGen 平台已经被社区广泛下载使用，催生了不少应用示例（如 ChatDev 软件开发代理等 [57]），表明其实用性与影响力在不断提升。

OpenAgents 与开源平台: 开源社区也在积极推动 Agent 平台的发展，其中 **OpenAgents** 是一个具有代表性的开放平台 [22]。OpenAgents 由学术与开源团队联合开发，目标是提供一个“一站式”的 Agent 托管与使用环境 [23]。它带来的一个关键进步是**让非专家用户也能便捷地使用 Agent**：通过一个网页界面，用户可以选择不同类型的 Agent（数据分析、插件工具、网页浏览），输入自然语言指令，即可获得 Agent 自动执行复杂任务的结果 [23]。例如，用户可以要求“帮我分析这份表格数据”或“在网上预订最近评价好的酒店”，OpenAgents 会调度相应 Agent（如Data Agent调用Python分析，Web Agent在浏览页面）并在界面上返回结果。OpenAgents 平台还注重**开发者部署体验**，提供了在本地或服务器快速部署Agent服务的方案，从而方便集成到自己的产品中 [26]。在生态上，OpenAgents 将三种常用能力（数据、API插件、网页）封装为标准 Agent，并通过开放接口支持社区开发更多 Agent 插件。这种模式下，OpenAgents 有望成为一个开放的 Agent 集市，让各种专用Agent都能在统一平台上被发现和使用 [23]。

WebAgent 与 AgentVerse: 某些平台专注于特定领域的 Agent，如 **WebAgent** 通常指专门面向网页浏览和操作的智能体。例如 OpenAgents 中就内置了一个 WebAgent，可在**真实网页环境**中自主执行搜索、点击等操作，以完成给定任务 [24]。类似地，Meta 的 WebGPT、Google 的 BrowserGPT 等也可视为 WebAgent 的变体，它们通过让 LLM生成浏览器操作序列，实现在互联网页面上的信息获取与交互。另一方面，**AgentVerse** 是一个着重于**多智能体协作与环境模拟**的开源框架 [58]。AgentVerse 支持两种模式：任务求解和社会模拟 [59]。在任务求解模式下，它可以**编排一组专家 Agent**共同完成一个任务，每个 Agent 执行不同子任务并共享信息；在模拟模式下，它可以让多个 Agent 在一个虚拟环境中交互，以研究群体行为的涌现 [60]。AgentVerse 强调 Agent 组成的灵活调整和协作收

益，通过实验展示了多Agent组合能够超过单Agent的性能 [58]。例如在一个协同任务中，多Agent组相比最强的单Agent取得了更高效率。这表明在工业应用中，引入多Agent团队有潜力提升复杂任务的完成效果。AgentVerse 当前在研的方向包括如何利用多Agent模拟人类团队，提高决策的合理性，以及探究多Agent产生的**社交行为**及如何加以引导 [58]。

ChatDev 与 MetaGPT: 工业界和开源社区近期涌现了一批模拟专业场景的 Agent 系统，其中以软件开发为代表。**ChatDev** 是一个开创性的尝试，它模拟了一家由多 Agent 构成的虚拟软件公司 [61]。ChatDev 框架下，不同的智能Agent被赋予软件公司中的典型角色（如CEO、CTO、程序员、测试员等），它们协同运作，分工完成软件开发流程 [62]。例如，CEO Agent 负责总体规划和验收，程序员 Agent 根据需求编写代码，测试 Agent 检查代码问题，最终各角色Agent合作产出一个软件 [61] [63]。整个过程由系统自动协调，各 Agent 通过对话举办"会议"共享进展 [63]。ChatDev 展示了 LLM 驱动的多Agent如何实现复杂的**创造性工作流**，其开源实现也让开发者可以定制类似的多Agent团队用于自己的项目。**MetaGPT** 的思路与 ChatDev 相近，同样将软件开发过程分解给多个专业 Agent 来模拟一家 AI 软件公司 [64]。MetaGPT 由 DeepWisdom 开源，强调结合人类的软件工程知识和 AI Agent 的自动化来完成任务 [64]。在 MetaGPT 中，有的 Agent 扮演产品经理撰写需求文档，有的Agent扮演架构师设计模块划分，有的负责编码实现，仿照真实企业的协作模式 [64]。MetaGPT 在 GitHub 上迅速流行，获得了大量关注和星标，表明业界对这种 **"多Agent模拟真实业务"** 的模式非常感兴趣 [65]。ChatDev 和 MetaGPT 的成功说明，在复杂任务（如软件开发、游戏设计等）中，引入多智能体的专业分工协作，可以大幅提高自动化水平。这类系统也为企业提供了新的自动化方案：未来也许一个项目经理只需下达一个高层指令，Agent团队即可产出完整的软件产品。

平台特性对比: 当前主流 Agent 系统在功能特性上各有侧重。一些平台支持**低代码/可视化**构建，如前述 AutoGen Studio 提供了拖拽式界面方便设计多Agent流程 [66] [67]；一些系统强调**Agent 即服务**，允许一键部署为 API，如 AutoGen、LangChain Agent 等都支持将 Agent 工作流封装为可调用服务 [52]。在框架基础方面，许多工业系统都基于成熟的开源框架。例如 **LangChain** 提供了丰富的 Prompt 模板、工具封装和 Agent 执行控制，被广泛用作底层支撑；**LlamaIndex (GPT Index)** 则专注于**检索增强**场景，为 Agent 提供向量数据库接入和文档检索能力，用于 AgentRAG 模式下的信息获取 [68]；**LangGraph** 前面已述，是 LangChain 官方推出的新框架，为多Agent工作流提供了更强的可控性 [19]。此外，社区还有 **Haystack** (侧重问答Agent)、**FrugalGPT** (关注成本优化)、**HuggingGPT** (使用ChatGPT调度专家模型) 等不同侧重的框架。不久前出现的 **CrewAI** 则定位为一个易用的多Agent编排框架，强调"低门槛"，内置了很多企业常用模板 [68]。总的来说，工业界平台的发展方向一是与开源生态深度融合（充分利用像 LangChain 这样的积木），二是打造 **端到端解决方案**（从构建、调试到部署的一站式支持），三是探索**新模式**（如 Agent 市场、可视化编排等）。这极大降低了企业将 Agent 引入实际业务的门槛。

(表：主流开源 Agent 框架比较)

图：*LangGraph*、*LlamaIndex*、*AutoGen*、*CrewAI* 等开源框架的能力对比表，展示了各框架在代理类型、执行逻辑、工具集成、人类参与等方面的差异 [68]。例如，*LangGraph* 和 *AutoGen* 支持多*Agent*对话流程，*LlamaIndex* 专注检索增强型*Agent*，*CrewAI* 强调用户友好和模版提供。该比较有助于企业根据需求选择合适的*Agent*开发框架。

4. 协议与标准 (Protocols and Standards)

MCP (Model Context Protocol): 随着 Agent 系统愈发复杂，业界开始制定标准以规范模型与工具、模型与模型之间的交互。其中，由 Anthropic 发起的 **MCP** 是一个重要的开放标准。MCP 的作用是充当模型与外部服务之间的**上下文通信层**，提供一套标准接口让 AI 模型能够安全、统一地调用工具、数据库或模板等外部资源 [69]。直观类比来说，MCP 之于 AI 应用，就像 **USB-C 接口** 之于硬件设备 [70] [71]——它定义了统一的"插槽"和协议，使各种工具和数据源可以以标准化方式接入 AI 模型 [71] [72]。通过 MCP，Agent 可以在需要某项功能时，根据标准协议请求对应服务，并将结果纳入自身上下文，而无需为不同服务定制不同集成代码 [73] [74]。例如，一个支持 MCP 的 Agent 要

查询数据库，只需按照 MCP 定义的格式发送查询请求，MCP 层将与数据库交互并返回结果，Agent 再将结果纳入后续对话。MCP 的出现解决了 **工具接入碎片化**的问题，让新增工具变得如同插入USB设备一样便捷 [72]。Anthropic 于 2024 年发布了 MCP 规范，并开源了参考实现 [75]。如今，包括 Feishu (飞书) 等在内的多家厂商已开始在自家产品中支持 MCP。例如，飞书推出了 MCP 服务，使 IDE 开发插件（如 Cursor、Windsurf）能够通过 MCP 访问飞书文档和项目管理等数据 [76] [77]。通过 MCP，飞书里的知识库、待办事项等可被开发者AI助手安全地读取和操作，形成了企业内部"Agent 接入企业数据"的范例 [76]。值得注意的是，MCP 并不是某种 Agent 实现框架，而是一个**集成层协议** [78]。它关注于"如何标准地连通"，而不涉及"何时做什么"。因此 MCP 常与具体的 Agent 框架配合使用：比如 LangChain、LangGraph、LlamaIndex 这些框架可以通过 MCP 去调用外部工具，从而提高兼容性和可维护性 [78]。总的来说，MCP 的推广为 Agent 生态提供了一个**统一接口**，让模型与工具之间的交互更高效、安全和结构化 [79] [73]。随着更多厂商和开源项目支持 MCP，Agent 将更易于扩展能力、接入企业内部各种数据源，进而变得更加实用。

A2A (Agent-to-Agent Protocol): Agent 间通信也是关键一环。为此，Google 在 2024 年推出了 **A2A 协议**，即 Agent 间通信协议 [80]。A2A 定义了一套标准框架，使得不同开发者创建的 AI Agent 可以**用一致可预测的方式互相通信** [80]。简单来说，如果 MCP 关注"模型对工具"的接口，那么 A2A 就关注"Agent 对 Agent"的语言 [81]。在没有 A2A 之前，不同 Agent 想协作往往需要定制集成（例如一个旅游计划Agent如何调用一个航班预订Agent，各自接口不统一）。A2A 的出现提供了**统一的通信格式和流程**，使多个 Agent 可以自动发现彼此能力、交换信息、协同处理任务 [82]。具体特性方面，A2A 包括：**Agent 卡片**（Agent 的标准化能力描述） [83]、**通信信道**（如 JSON over HTTP 的消息格式）、**错误处理**（规范Agent在协作中如何报告失败）等机制 [84] [81]。通过 Agent Card，一个 Agent 可以向网络中其他Agent公告自己的名称、功能、提供的API端点等 [85] [86]。这样，当一个任务需要多个能力时，Agent 可以根据卡片发现哪个Agent能帮忙，然后通过标准消息协议向其发送请求并获取结果。A2A 协议推出后，得到了包括 Intuit、LangChain、MongoDB 等公司的支持和采用 [87]。这意味着，一些主流Agent框架（如 LangChain）已经内置兼容 A2A，可以让LangChain创建的Agent与外部Agent对话协作 [88]。需要强调的是，MCP 与 A2A 针对不同层面：前者管"Agent 和工具对话"，后者管"Agent 和Agent对话" [81]。两者可以同时用于一个复杂系统中，例如一个场景下，两个 Agent 各自通过 MCP 获取所需信息，然后彼此通过 A2A 协商决策 [89] [90]。除了 Google 的A2A，IBM 也贡献了类似的 **ACP (Agent Communication Protocol)** [91] [92]。IBM 的 ACP 以 Linux 基金会开源项目 BeeAI 形式发布，采用 RESTful HTTP 协议，让 Agent 之间可以点对点直接通信 [92] [93]。ACP 被设计为与 MCP 互补：MCP 连通工具，ACP 连通 Agent [92]。通过 ACP，Agent 可以在无"中央指挥"的情况下平等交互，实现**去中心化**的协作架构 [94]。例如，一个客服 triage Agent 接待客户后，可将对话上下文通过 ACP 转交给合适的专职服务Agent继续处理 [95]。这种模式下Agent彼此没有上下级之分，可根据需要灵活启动对话。IBM 和 Google 的双协议并行表明，行业对于 Agent 通信标准非常重视，多个方案百花齐放。未来可能会出现 MCP 与 A2A 的融合或互操作标准，让不同厂商的 Agent 网络实现无缝协同。对企业而言，这意味着可以采购和部署不同来源的智能 Agent，并让它们在统一规范下**协同工作**，共同完成复杂业务流程。

应用模式: 有了标准协议后，Agent 的应用模式也更加灵活多样。例如，通过 MCP，一个企业可以让自己的内部 GPT助手直接读取公司知识库、数据库，从而给出基于最新内部数据的回答，而无需担心定制接口的问题 [72]。通过 A2A，不同部门的AI代理（如销售Agent、客服Agent）可以在获得客户请求时相互协商，把任务自动转交给最合适的Agent处理 [94] [95]。这类似于在企业内部搭建一张**AI Agent 网络**：每个Agent各司其职，又能相互通信配合，最终提供端到端的自动化服务。当然，实现这一切的前提是协议本身得到广泛支持和安全可靠。因此，当前业界也在积极完善这些标准的安全机制（如权限管理、身份认证等）。总之，MCP 和 A2A 为 Agent 生态带来了**标准化的语言**，如同互联网时代的 TCP/IP 协议，大大促进了不同系统间的互联互通，对于 Agent 技术从单点应用走向规模化协作具有里程碑意义。

5. 多智能体系统 (Multi-Agent Systems)

多 Agent 调度策略: 在许多复杂场景下，单个 Agent 的能力和效率可能不足，因此引入**多智能体系统**让多个 Agent 合作或竞争完成任务成为一大发展方向。在多 Agent 系统中，如何调度这些 Agent 是核心问题之一。常见的多 Agent 调度/组织策略包括：

- **主从式 (Master-Agent) 架构：** 这是最直观的架构，由一个**主代理（Master）**负责总体协调和决策，将子任务分配给一个或多个**从代理（Worker）**去执行 [96]。主代理通常具备全局视野和规划能力，而从代理是专门的执行者。比如在 HuggingGPT 框架中，就由一个中央的 ChatGPT Agent 作为控制器，分配任务给各个专业模型（图像识别模型、语音模型等）完成 [97]。又如在 AutoGPT/BabyAGI 等系统中，常设置一个 Manager Agent 反复读取目标和当前状态，然后创建/调用 Executor Agent 执行具体操作。Master-Agent 模式的优点是**结构清晰**，便于控制流程、汇总结果，也易于在出现错误时由主代理进行纠偏 [94]。许多企业应用采用此模式：例如一个调度 Agent 根据用户请求决定调用哪个部门的AI助手，再整合它们的输出。这类似于现实中的项目经理协调团队。但缺点是主代理成了单点，一旦其失误或性能瓶颈，会影响整体效率。

- **投票/评议式 (Voting/Ensemble) 策略：** 该策略让多个 Agent **并行地提出解决方案或观点**，然后通过投票或评议机制选出最佳结果 [98] [99]。这种方法提高了可靠性和准确性，尤其在需要判断正确性的任务上。例如，LLM-Debate 是 OpenAI 提出的让两个或多个语言模型 Agent 辩论某问题，并由一个评判模型决定胜方，从而输出答案的方法 [37]。又如 **CoT-SC (Chain-of-Thought with Self-Consistency)** 方法中，让模型对同一问题生成多条思考链，然后选择多数链条达成一致的答案作为最终答案 [100]。实践证明，这种投票表决可以有效**降低单一路径错误**的概率 [38]。在决策场景中，也可以有多个 Agent 提供各自方案，再由系统综合评分或投票决定执行哪一个方案 [101]。Voting 策略的优势在于**容错性**和**稳健性**更高，就像群体决策往往优于个人。缺点是需要更多计算资源（并行运行多个Agent），而且仍需一个公正的投票/评议机制来汇总结果。

- **去中心化 (Decentralized) 架构：** 在这种模式下，没有固定的中央控制 Agent，各个 Agent **对等 (Peer-to-Peer)** 地交互，整体通过某种协议达到协作目标 [94]。Agent 可以彼此发现对方、通信协商、互相发送任务请求，类似一个自治的群体。例如 IBM 的 ACP 协议和 Google 的 A2A 协议就是为了支持去中心化的多 Agent 协作而设计 [92] [88]。在这种架构中，可能某个Agent发起任务后，通过广播或查询找到能处理该任务的其他Agent，再建立临时协作关系完成任务。这样的好处是**灵活性**高，系统健壮性也更好（不存在单点故障），可以根据需求动态扩展Agent节点 [94]。举例来说，在一个企业AI系统中，一个客服Agent处理到一半发现客户询问财务问题，它可以直接通过Agent网络找到财务Agent，并将对话上下文传递过去，由财务Agent接手 [95]。整个过程不需要一个中央调度，一切由Agent间协议自动完成。这种架构的挑战在于设计有效的通信和信任机制，防止Agent间信息遗漏或冲突。此外，去中心化系统有可能出现决策延迟（因为需要协商）或者局部的次优解（没有全局视野）。为此，常用的技术包括 **黑板系统 (Blackboard)**（Agent 通过共享内存交流信息）、**合同网协议 (Contract Net)**（通过投标竞价分配任务）等，来协调多Agent的自主互动。

多 Agent 通信机制: 无论哪种调度策略，多智能体系统都需要高效的通信机制。常见的通信方式有：

- **直接消息传递：** Agent 彼此通过预定格式的消息进行点对点通信。这种方式类似发送邮件或API请求，需要有共同的通信协议（如前述 A2A）。直接消息可实现询问-回答、通知、商议等各种对话模式。例如两个 Agent可以交换JSON消息，一个发送请求 `{"task": "lookup", "query": "天气"}`，另一个返回结果。这种方法实现简单，但要求 Agent 对消息语义有统一理解。A2A 和 FIPA-ACL 是两种广为人知的 Agent 通信语言标准，使不同实现的Agent也能互通 [88]。

- **共享内存/黑板 (Shared Memory/Blackboard):** 多个 Agent 通过访问**公共的共享数据结构**来交换信息。如黑板模型中，设立一个全局黑板数据区，Agent 可以在上面读写信息，相当于留言板 [102]。当一个Agent获得新的部分解或数据，就发布到黑板，其他Agent监视黑板上的变化，看到自己需要的信息就做出响应。共享内存机制的好处是松耦合，Agent 不需要直接知道对方，只通过共享空间通信。这在**多Agent调度**或**分布式计算**中常用。但是要注意并发冲突和数据一致性问题，需要有仲裁机制避免多个Agent同时修改数据的冲突 [103]。
- **语言通信：** 特别是在 LLM 时代，一种有趣的模式是 Agent 之间通过自然语言对话交流，而不是结构化消息。例如 CAMEL 框架中，让两个 ChatGPT Agent 分别扮演用户和 AI，不断对话来推动任务完成。这种通信利用了LLM对语言的理解能力，方便以开放式讨论的形式解决问题。不过语言通信可能产生歧义，而且效率不如结构化数据。在实际系统中，可将其与上述方法结合：即Agent产生自然语言，再包装在消息中发送。

共享记忆机制: 多 Agent 有时需要**共享记忆或知识**，以避免各自为政、重复劳动。实现共享记忆有几种方式：

- **全局知识库：** 建立一个所有 Agent 都可以查询的知识库或数据库，里面汇总了各Agent获取的重要信息。例如，在一个科研助理Agent系统中，各Agent查过的论文摘要可以存入一个全局向量数据库，后来的Agent在回答相关问题时可复用这些摘要，而不必重复查找。这样的知识库充当集体记忆，每个Agent既是贡献者也是使用者 [104]。Memory Sharing (MS) 框架就是类似思想，它让多个Agent将各自交互中产生的新知识不断加入共享"记忆池"，从而集体受益 [105] [104]。研究表明，共享记忆可以提升多Agent系统的整体表现，使其朝"集体智能"演进 [104]。
- **发布订阅机制：** Agent 可以将自己获取的某类信息发布出去，有需要的Agent会订阅该信息类型。比如一个环境监测多Agent系统中，摄像Agent发布"检测到车辆"事件，交通控制Agent订阅此事件后触发相应动作。这种模式保持每个Agent独立运行，通过事件系统松散耦合地共享状态。
- **模型共享：** 在一些场景下，多Agent可能共享底层的AI模型参数或部分模块。例如，同质的巡逻机器人Agent队伍可以共享同一个导航模型，每个Agent同时更新这个模型（类似联邦学习），达到共享学习的目的。这也是一种共享记忆（模型参数即长期记忆）。

任务协同方式: 多智能体如何协同完成任务，主要取决于任务的可分解性和交互模式。典型的协同方式包括：

- **任务分解 (Task Decomposition)：** 将复杂任务拆解为可并行或串行的子任务，由不同Agent承担，再汇总结果。例如自动数据分析任务可以分成"数据清洗"、"统计计算"、"图表生成"三部分，由三个Agent并行完成后将结果合并成报告。任务分解需要一个规划者Agent或预先规则来确定子任务划分和执行顺序。
- **角色分工 (Role Specialization)：** 类似ChatDev、MetaGPT，把不同职责分配给不同Agent，按流程由一个Agent的输出成为下一个Agent的输入。例如在软件开发中，需求文档->设计文档->代码->测试，每一步由特定角色Agent负责，像生产线一样衔接。这种方式适合流程明确的任务。
- **冗余验证 (Redundancy & Verification)：** 如前述投票策略，让多个Agent独立产生解答，然后相互核验。比如关键任务（医疗诊断等）可设置两个Agent分别诊断，再由第三个Agent比较结果，若不一致则协调重新评估。这种方式提高可靠性。
- **竞争与博弈 (Competitive/Adversarial)：** 在某些场景下引入Agent间竞争可以优化结果。例如两个Agent辩论一个问题，迫使彼此提出更有力的论据，最终评判者得到更完善的答案 [37]。又如博弈环境（棋类、拍卖等）中，多Agent竞争是问题本身的一部分，需要设计奖励机制鼓励正确的竞争策略。
- **去中心协同 (Emergent Collaboration)：** 在没有全局控制的情况下，多Agent通过本地交互涌现出协作行为。比如无人机编队，每个Agent遵循简单避碰和跟随规则，但整体出现队形维护、目标围捕等复杂协作。这通常需要借助强化学习或进化算法，让协作策略在模拟中自行涌现。

总的来说，多智能体系统提供了远超单Agent的灵活性和潜力。但也带来了新的挑战，如**协同决策的复杂性**、**通信开销**、**策略收敛**以及**潜在冲突**等。这些正是多智能体系统领域（MAS）长期研究的问题。近年来，LLM的加入为

MAS注入了新的活力，因为LLM擅长语言沟通和综合规划，在多Agent协作中可以担任“智慧中枢”或“交流媒介”的角色。例如，有研究让一个LLM Agent担任多机器人团队的指挥官，用自然语言给每个机器人下达指令，取得了比传统规划更好的柔性。这些探索预示着未来的多智能体系统将融合数据驱动的智能和符号通信协议，朝着更加聪明、高效的群体智能方向发展。

6. 典型场景与落地案例 (Applications and Case Studies)

网页浏览 Agent: 让 Agent 充当“数字浏览器”替人执行网页操作，是一个重要且实用的场景。在这个领域，OpenAI 曾推出 **WebGPT**，通过强化学习让 GPT-3 系统学会了询问用户需求、检索网络、打开浏览结果并汇总答案，以回答开放域问题 [106]。WebGPT 证明了大型语言模型可以控制浏览器完成信息查询和综合。之后，出现了如 **BrowserGPT、WebRunner** 等一系列浏览 Agent，它们通常使用 ReAct Prompt 或工具调用，让 LLM 输出诸如“Search(query)”、“Click(link)”等指令序列，驱动无头浏览器执行 [106]。学术界的 WebArena 基准 [42] 进一步推动了研发，在模拟真实网站环境中评测 Agent 预订航班、网购等复杂操作的成功率 [42]。这些系统和基准显著提高了 Agent 的网页操作能力。在工业落地方面，已经有产品将浏览 Agent 应用于日常。例如微软 Bing Chat 引入了浏览模式，当用户提问时，Agent 可自动搜索网络并阅读网页以提供最新答案（如实时新闻摘要）。一些自动化办公工具也使用浏览 Agent 来执行跨系统的重复性任务，比如填报网页表单、下载报表等，解放人工操作。可以预见，随着 Agent 安全性和可靠性提升，**Web 浏览 Agent** 将在企业信息搜集、电商导购、网络监测等领域获得广泛应用，让用户只需提出需求，Agent 就能遍历互联网为其完成目标。

GUI 操控 Agent: 除了网页，Agent 还被用于操作图形用户界面 (GUI) 的软件和系统。这类 Agent 可以看作是通用的“机器人流程自动化 (RPA)”的智能升级版。传统RPA依靠预录宏或规则完成鼠标点击、键盘输入等操作，而 GUI Agent 则利用视觉理解和语言指令，能够自主探索界面完成任务。比如，GitHub 有项目尝试让 GPT-4 根据界面截图和文本描述，生成对系统UI的操作序列，实现自动化软件测试。又如微软研究的 **TaskMatrix.AI** 计划，旨在让一个通用Agent可以跨App完成操作：用户用自然语言下达指令，Agent 通过OCR读取应用界面内容，决定点击哪个按钮、输入什么文本，以跨应用地完成复杂任务 [107] [108]。在实际案例中，已有初创公司将此用于自动化办公助手：员工可以对Agent说“帮我把这100封邮件附件下载保存，然后把这些PDF合并成一个”，Agent 就会像人一样操作邮件客户端和PDF工具完成任务。目前这方面的技术挑战在于界面变化多、元素识别和定位难，但随着计算机视觉和UI元数据的结合，GUI 操控Agent的可靠性在提高。一些IDE插件（如 Cursor AI）也引入Agent在IDE界面执行操作、调试代码等。总的来说，GUI Agent 让通用软件使用自动化成为可能，对于减少人工重复劳动、提升效率具有巨大潜力。

DevAgent（开发者代理）： 软件开发是Agent大显身手的另一典型场景。前述 ChatDev、MetaGPT 即是这种 **DevAgent** 的代表案例：它们通过多Agent分工，几乎可以全流程自动完成一个软件项目 [61] [64]。在实际商业中，也有公司推出面向开发团队的 AI 助手。比如 **GitHub Copilot X** 增强版引入了一个任务Agent，可以协助开发者生成项目模板、管理拉取请求等；再如 **AWS CodeWhisperer** 主要生成代码，但AWS内部据传也有代号“CodeGuru”的Agent，用于自动代码审查和性能分析。更激进的是 **CloudApper 的 AI DevAgent™** [109] 等产品，宣称Agent可承担从编码、测试到部署的许多开发任务，充当团队的24/7开发同事 [109]。这些DevAgent通常集成在IDE或DevOps流程中，通过解析项目文档、监控issue列表等获取环境信息，然后执行相应开发任务（例如自动修复一个常见bug、生成单元测试、优化配置等）。此外，学术界也探讨了 Agent 辅助软件工程的方向，例如让Agent参与需求分析（从用户故事中抽取需求列表），或者在多人协作时充当智能协调者（自动创建待办事项、提醒进度等）。可以预见，将来DevAgent有望**减少程序员样板工作**、**加速迭代**，甚至可能催生新的开发模式（人类关注创意和高级逻辑，Agent处理具体实现）。当然，目前DevAgent还无法完全替代人，但已经能够显著提高生产力，对企业而言具有很高的投资回报。

AgentRAG（Agent + 检索增强生成）：在企业知识问答、科研助理等场景，经常需要 Agent 具备实时获取外部知识的能力。**AgentRAG** 指的是结合检索增强生成 (Retrieval-Augmented Generation) 技术的智能Agent：Agent 一方面利用 LLM 强大的语言理解生成能力，另一方面通过检索模块获取最新的或领域专门的信息。这类系统通常工作流程是：接受到用户任务后，先由Agent分析出需要查询的关键点，调用**搜索Agent或数据库Agent**获取候选资料，然后将资料融入上下文，再由LLM生成回答或方案。例如，一个法律咨询Agent遇到复杂问题时，可以自动检索法律法规文本，将相关条款提供给LLM参考，从而给出准确答案；又如一科研助理Agent可以查询学术论文数据库，找到相关研究结论后再撰写总结报告。这种 AgentRAG 思路已经在诸多产品中应用：Microsoft Bing Chat就是典型，它每次回答都会背后实时搜索网页作为引用；Notion AI 引入自有知识库检索，使其能基于用户私人笔记内容回答问题；企业内部的智能助手也普遍加入了对公司Wiki、SharePoint等的检索接入，让回答更加定制化准确。开源框架方面，LlamaIndex (原GPT Index) 为 AgentRAG 提供了便捷工具，将文档集编入向量索引，Agent 查询时能快速得到相关段落，避免了LLM碎片记忆的弱点 [110]。总之，AgentRAG 通过 **让Agent具备查询记忆外知识的能力**，大幅拓展了应用范围。从客服、文档问答，到市场调研报告生成，凡是需要把散落在各处的信息整合起来的任务，都是AgentRAG大显身手的地方。它也有效缓解了LLM闭域模型"知识截至"的问题，使Agent始终掌握最新最全的信息 [106]。

科研助理与教育机器人: 在专业领域，Agent 逐渐扮演"智能助理"角色。例如在科研领域，语言模型Agent可用来梳理文献、生成研究灵感。已经有研究人员利用Agent帮忙检索特定课题下最新论文，并让Agent提炼这些论文的核心观点、比较差异，形成文献综述初稿。这相当于一个科研助理帮你读大量文献并摘要 [36]。一些工具（如 Elicit）已经将LLM Agent用于半自动文献调研：用户输入问题，Agent 会检索论文数据库给出相关研究的结论、方法比较。此外，Agent 还可用于实验设计辅助、数据分析建议等科研环节。在教育场景， **教育机器人/辅导Agent** 也在兴起。比如微软亚洲研究院的 **MentorChat** 原型，让一个多模态Agent观察学生的解题步骤，并给予提示和反馈，犹如私人家教。还有公司推出面向儿童的对话Agent，可不断学习孩童的兴趣与知识水平，提供量身定制的解答和引导。教育Agent 的优势在于 **个性化**：每个学生都相当于拥有一位全天候私人教师，可以随时问问题、获取练习批改和学习建议。这在过去是不可想象的。尽管目前Agent的教育水平有限，但已能胜任基础的答疑和陪练。随着知识图谱、学生模型结合LLM，未来教育机器人将更深入地理解教学规律，发挥更大作用。

企业部署案例: 越来越多的企业开始将 Agent 整合进业务流程，实现"AI 同事"或"AI助手"。以下是几个实际落地的案例：

- **AWS QuickSight Q:** Amazon QuickSight 是 AWS 的商业智能 (BI) 服务，其中的 "QuickSight Q" 功能引入了自然语言问答 [111]。用户可以用日常语言询问类似"本季度销售额同比增长是多少？"，QuickSight Q 的 Agent 就会在后台理解问题意图，自动检索数据源并计算答案，用图表或数字形式回答 [112]。这是典型的企业Agent，用于数据分析领域。它将过去需要BI专家写SQL或Excel公式的工作，转化为由Agent自动完成，让业务人员直接对话BI系统获得洞察 [111]。QuickSight Q 成功将 Agent 融入数据分析这一垂直场景，大大降低了数据洞察的门槛。
- **飞书 (Feishu) MCP 应用:** 飞书是字节跳动旗下的企业协作平台，近期它积极拥抱 Agent 技术。在飞书的产品中集成了一个"AI 助手"，可以帮助 summarizing 文档、翻译对话、生成日程等。而其背后的关键在于使用了 **MCP 协议** 将飞书内部各类应用接口开放给AI Model [76]。例如，开发者IDE插件 Cursor 通过 MCP 服务器访问飞书文档系统，实现了编程时AI助手可直接查询项目文档、需求列表等 [76]。又如，飞书基于 MCP 开发了项目管理助手，Agent 可以通过统一协议读取项目进度、更新任务状态 [77]。这些应用展现了 MCP 在企业内的实际价值：各部门工具（文档、日历、任务管理）只需提供统一描述，就能被企业AI助手使用。飞书的Agent化实践提升了办公协同效率。例如，在飞书中@AI助手请它"整理会议纪要并创建任务"，Agent 会读取会议文档要点，然后调用任务管理API自动创建待办，完成过去需要多人配合的工作。这为其他国产协同软件树立了标杆。
- **Notion AI:** Notion 是流行的笔记和协作工具，其内置的 **Notion AI** 是一个面向终端用户的 Agent 集成案例。Notion AI 可以帮用户撰写笔记初稿、头脑风暴大纲、翻译内容，甚至直接根据笔记内容回答问题

[113]。这背后使用了OpenAI的GPT模型并辅以Notion自有的数据检索：当用户让AI总结某个项目空间的信息时，Agent 会访问该空间下的所有笔记和数据库，以获取事实，再撰写总结。这种**嵌入式Agent** 极大增强了生产力工具的能力 [113]。Notion AI 的出现使知识工人在一个软件内就能完成从内容生成到信息查询的一站式工作流，无需频繁切换应用。由于 Notion 平台的开放性，AI Agent 得以无缝融入，成为用户工作空间中的“数智助手” [113]。据官方统计，引入AI后用户撰写文档的效率提高了数倍，Notion 也借此在竞争激烈的协作软件市场中树立了差异化优势。

- **Microsoft 365 Copilot:** 作为重量级案例，微软在 2023 年宣布将 GPT-4 Agent 集成到 Office 全家桶，称为 Microsoft 365 Copilot。这实际上是在 Word、Excel、PowerPoint、Outlook 等应用中引入了上下文感知的AI助手。例如，在Word里，Copilot可以基于文档内容自动起草剩余章节；在Excel里，它可以根据数据生成透视分析报告；在Outlook里，它能阅读邮件并起草回复。Copilot 利用了微软图谱中的企业数据并结合用户实时文档，实现了深度**上下文整合**。这一产品将 Agent 真正引入了主流办公流程，预计会极大改变白领的日常工作方式。虽然用户在隐私、安全等方面有所顾虑，但微软通过企业级权限和数据隔离等手段，努力让Copilot成为可信赖的Office助手。
- **其他案例:** 还有许多值得一提的应用：Salesforce 推出的 **Einstein GPT** 将Agent嵌入CRM，帮助销售自动撰写跟进邮件、生成销售预测；SAP 的 **Virtual Copilot** 则集成在ERP系统中，帮助财务人员用自然语言查询业务数据；国内阿里的 **通义千问** 已开放插件功能，企业可将自己的服务接入大模型Agent，打造定制客服和导购机器人。这些案例都显示了**大模型Agent正在从概念走向商业价值**：它们被部署在真实业务环境中，和人类员工共同协作，提升效率、降低成本，并创造新的用户体验。

总的来说，企业实际部署Agent主要出于两点考虑：一是**提升内部效率**（如自动化数据分析、代码管理等），二是**改善客户体验**（如更智能的客服、更贴心的办公助手）。随着技术成熟，我们预计几乎每个知识工作者都将拥有一个AI Agent助手，每个企业系统也会内嵌AI Agent支持。这将像当年办公软件普及一样，带来生产力的新飞跃。

7. 未来趋势与未解决问题 (Future Trends and Challenges)

Agent 向“主动计划者”演化: 当下许多 Agent 仍属于被动回应型，即接到用户请求或触发条件后才行动。未来一个重要演化方向是让 Agent 成为**主动计划者**，能够在无人督促下连续开展工作，甚至自行发现和制定新任务。这类似从“工具”进化为“同事”的过程。为此，Agent 需要具备更强的 **自主规划**和**长期决策**能力。目前的尝试包括“Plan-and-Execute”模式，将任务规划与执行明确分开 [114]。Agent 先生成一个完整的高层计划（例如GPT-4给出解决问题的步骤清单），再逐步执行并根据需要调整。这种方式使得Agent对全局目标有了概念，不再只是一步步贪心地往前 [115]。OpenAI 的 AutoGPT/BabyAGI 原型正是这种思路的雏形：它们在循环中让Agent审视自己当前的目标列表，计划接下来要做的事，自我反馈，直到达到最终目的。虽然初版AutoGPT曝露出很多问题（例如容易卡住循环、自我重复等 [116]），但其展示了Agent主动性的可能性。展望未来，随着研究者在规划算法和LLM结合上取得进展（比如引入经典AI的规划算法PDDL、树搜索等），Agent 将逐步掌握**长时序、多阶段**任务的规划执行。它可以像项目经理一样，在一天开始时列出待办，在过程中动态调整顺序，并在完成后总结经验。这种主动性将极大拓展Agent应用的深度：例如个人生活助理Agent可以持续帮你规划日程、提醒、处理琐事，而不只是回答问询。

Agent + Planner 联动: 为实现上述目标，一条明确路径是结合专用规划器（Planner）与语言Agent的优势。LLM擅长开放式推理和语言生成，但在严谨的计划搜索上不够完备；相反，传统规划算法（如经典AI的A搜索、约束规划）在穷尽搜索和保证最优解上很强。两者联动可以取长补短。近期有工作探索了*“LLM + 搜索算法”*的融合，例如华人学者提出的 *GPT-Plan*，让*LLM*产生问题的高层抽象计划，再用约束求解器验证和补全细节，最终执行。还有计算机视觉领域的 *LLM-Planner*，用于让机器人遵循自然语言指令完成复杂任务：其中*LLM*负责将任务分解成动作序列，再由经典规划算法在物理环境中寻找可行路径 [39]。结果表明，这种系统比单纯*LLM*或单纯规划算法表现更好 [39]。

我们预计未来会出现通用的 *PlanGPT* 或 *LM-Planner* 模块，专门作为*Agent*的大脑皮层，用于复杂任务的计划生成和评估。而*LLMAgent*更多扮演执行官和学习者角色，负责与环境交互和处理非结构化信息。两者通过共享记忆和反馈循环达到协同 [117]。比如，一个家庭服务机器人*Agent*可以使用内置的规划模块算出清洁房间的最优路线，用*LLM*模块处理自然语言指令和意外情况，两者结合后显著提升了完成任务的效率和可靠性。*Agent*和*Planner*的联动还有助于提升可解释性和安全性*——Planner产生的计划往往是结构化、透明的，人类可以检查审批，从而避免Agent走偏。这也是未来在重要领域应用Agent所必须的保障。

安全性挑战: 安全始终是Agent走向大规模应用的最大挑战之一。具体包括：**指令安全**（防止Agent被恶意指令利用）、**内容安全**（避免产生有害输出）和**行动安全**（阻止Agent采取危害系统或用户的行为）。已有研究揭示，Agent

---

[1] [2] [3] Is It an agent, or just a program?: A taxonomy for autonomous agents
https://course.ccs.neu.edu/cs5100f11/resources/franklin96.pdf

[4] (PDF) An introduction to Soar as an agent architecture - ResearchGate
https://www.researchgate.net/publication/228398068_An_introduction_to_Soar_as_an_agent_architecture

[5] Comparing four cognitive architectures: SOAR, ACT-R, CLARION ...
https://roboticsbiz.com/comparing-four-cognitive-architectures-soar-act-r-clarion-and-dual/

[6] Soar (cognitive architecture) - Wikipedia
https://en.wikipedia.org/wiki/Soar_(cognitive_architecture)

[7] The Belief-Desire-Intention Model of Agency - SpringerLink
https://link.springer.com/chapter/10.1007/3-540-49057-4_1

[8] #13: Action! How AI Agents Execute Tasks with UI and API Tools
https://huggingface.co/blog/Kseniase/action

[9] [10] [11] [28] [2302.04761] Toolformer: Language Models Can Teach Themselves to Use Tools
https://arxiv.org/abs/2302.04761

[12] Simulations of Evolving Embodied Semiosis: Emergent Semantics in ...
https://casci.binghamton.edu/publications/ees_web.html

[13] [PDF] Environment Mediated Multi Agent Simulation Tools .
https://ccl.northwestern.edu/2008/Arunachalam_etAl_EnvironMediatedMAS.pdf

[14] [15] [27] [106] AI Agents vs. Agentic AI: A Conceptual Taxonomy, Applications and Challenges
https://arxiv.org/html/2505.10468v4

[16] [17] [18] AutoGen: Enabling next-generation large language model applications - Microsoft Research
https://www.microsoft.com/en-us/research/blog/autogen-enabling-next-generation-large-language-model-applications/

[19] [20] [21] What is LangGraph? | IBM
https://www.ibm.com/think/topics/langgraph

[22] [23] [24] [25] [26] [2310.10634] OpenAgents: An Open Platform for Language Agents in the Wild
https://arxiv.org/abs/2310.10634

[29] [30] [31] arxiv.org
https://arxiv.org/pdf/2305.10601

32 33 34 35 36 Reflexion | Prompt Engineering Guide
https://www.promptingguide.ai/techniques/reflexion

37 38 100 More Agents Is All You Need
https://arxiv.org/html/2402.05120v1

39 LLM-Planner: Few-Shot Grounded Planning for Embodied Agents ...
https://arxiv.org/abs/2212.04088

40 41 42 43 44 45 46 47 A Survey of Agent Evaluation Frameworks: Benchmarking the Benchmarks
https://www.getmaxim.ai/blog/llm-agent-evaluation-framework-comparison/

48 57 Revolutionizing the Future: The Power of ChatDev and Microsoft's ...
https://medium.com/@nibniw/revolutionizing-the-future-the-power-of-chatdev-and-microsofts-autogen-7a70ab672a2a

49 AutoGen v0.4: Reimagining the foundation of agentic AI for scale ...
https://www.microsoft.com/en-us/research/video/autogen-v0-4-reimagining-the-foundation-of-agentic-ai-for-scale-and-more-microsoft-research-forum/

50 52 53 54 56 66 67 Introducing AutoGen Studio: A low-code interface for building multi ...
https://www.microsoft.com/en-us/research/blog/introducing-autogen-studio-a-low-code-interface-for-building-multi-agent-workflows/

51 Using AutoGen to build your AI Agents - Microsoft Community Hub
https://techcommunity.microsoft.com/blog/educatordeveloperblog/building-ai-agent-applications-series---using-autogen-to-build-your-ai-agents/4052280

55 AutoGen Blog - Microsoft Developer Blogs
https://devblogs.microsoft.com/autogen/

58 60 [2308.10848] AgentVerse: Facilitating Multi-Agent Collaboration and Exploring Emergent Behaviors
https://arxiv.org/abs/2308.10848

59 AgentVerse is designed to facilitate the deployment of ... - GitHub
https://github.com/OpenBMB/AgentVerse

61 62 63 What is ChatDev? | IBM
https://www.ibm.com/think/topics/chatdev

64 65 What is MetaGPT ? | IBM
https://www.ibm.com/think/topics/metagpt

68 110 A Tour of Popular Open Source Frameworks for LLM-Powered Agents
https://blog.dataiku.com/open-source-frameworks-for-llm-powered-agents

69 70 71 72 73 75 78 79 What is Model Context Protocol (MCP)? | IBM
https://www.ibm.com/think/topics/model-context-protocol

74 80 81 82 83 84 85 86 87 88 What is A2A (Agent to Agent Protocol)? | by Akash Singh | Apr, 2025 | Medium
https://medium.com/@akash22675/what-is-a2a-agent-to-agent-protocol-d2325a41633a

76 Feishu-MCP（飞书模型上下文协议工具） - 腾讯云
https://cloud.tencent.com/developer/mcp/server/11143

77 基于MCP（Model Context Protocol）协议的飞书项目管理工具 - GitHub
https://github.com/Roland0511/mcp-feishu-proj

[89] [90] [91] [92] [93] [94] [95] An open-source protocol for AI agents to interact - IBM Research
https://research.ibm.com/blog/agent-communication-protocol-ai

[96] [98] [99] [101] How to Build a Multi-Agent AI System : In-Depth Guide
https://www.aalpha.net/blog/how-to-build-multi-agent-ai-system/

[97] NEW HuggingGPT - One Model to Rule Them All (Is this AGI?)
https://www.youtube.com/watch?v=PfY9lVtM_H0&pp=ygUII2hmY293YWk%3D

[102] [103] Memory in multi-agent systems: technical implementations - AI/LLM
https://artium.ai/insights/memory-in-multi-agent-systems-technical-implementations

[104] [105] Memory Sharing for Large Language Model based Agents
https://arxiv.org/html/2404.09982v2

[107] [108] How AI Agents Are Redefining Enterprise AI Architectures
https://superbo.ai/beyond-llms-how-ai-agents-are-redefining-enterprise-ai-architectures/

[109] CloudApper AI DevAgent™ - Your New Development Teammate
https://www.cloudapper.ai/ai-devagent/

[111] [112] Unlocking the Power of AI-Driven Insights and Analytics with ...
https://northbaysolutions.com/blog/unlocking-the-power-of-ai-driven-insights-and-analytics-with-amazon-quicksight-q/

[113] Notion AI Agents - Relevance AI
https://relevanceai.com/agent-templates-software/notion

[114] What are Plan and execute agents? - PromptLayer
https://www.promptlayer.com/glossary/plan-and-execute-agents

[115] ReAct vs Plan-and-Execute: A Practical Comparison of LLM Agent ...
https://dev.to/jamesli/react-vs-plan-and-execute-a-practical-comparison-of-llm-agent-patterns-4gh9

[116] Auto-GPT is the start of autonomous AI and it needs some guidelines.
https://www.reddit.com/r/GPT3/comments/12lrh8t/autogpt_is_the_start_of_autonomous_ai_and_it/

[117] Literature Review on Task Planning with LLM Agents - Isamu Isozaki
https://isamu-website.medium.com/literature-review-on-task-planning-with-llm-agents-a5c60ce4f6de